claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_4849400f-9d3\agent-a1f95ba558e8cdc7f.jsonl`
- SHA-256: `0a720fcfc1ad6b8ba1ddbf8ecdefa2fbbab06fc8479117e3f015af88f0d1fe4b`
- Source modified: `2026-06-21T09:29:36+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_4849400f-9d3`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T09:27:54.587Z)

```
You are mapping OWNER-GATED decisions for the badminton-highlight-indexer ENGINE serving track
(project doc docs/COMPUTE_DECOUPLED_SERVING/README.md). Context: M0-M4 are SHIPPED (the
config/input/output/device SEAM layer: deployment.profile+presets, input_backend, output base,
DeviceContext). NOW the team is on M5+. The owner wants to BATCH-DECIDE every gate so the
maximum independent code path is unblocked. The discipline: ONE isolated PR per milestone off
origin/master, default-OFF + golden/cross-OS-gated for any core-caller change, mocked tests for
anything not runnable locally (no GPU/cloud here). KEY DISTINCTION you must draw for every gate:
separate (a) CODE that CAN be written + merged NOW behind default-OFF with mocked/faked tests
(no owner needed) from (b) the precise OWNER PREREQUISITE (real GPU run, GCP spend,
counsel/consent review, real prices, OAuth creds, a design pick). For each gate return the
structured fields. Ground every claim in real file:line or doc anchors ? read the code, do not
guess. Be exhaustive within your area but do not stray outside it.

You are the SYNTHESIS step. Below are structured gate findings from 5 grounded readers (JSON).
Merge them into ONE prioritized DECISION MENU for the owner.

FINDINGS:
[
  {
    "area": "Compute-Decoupled Serving ? milestones M5?M12 + still-open §8 design questions
(docs/COMPUTE_DECOUPLED_SERVING/README.md §7 tracker, §8, §2 D7?D15; TESTING_STRATEGY.md;
ADR-014)",
    "gates": [
      {
        "id": "m5-truly-local-e2e",
        "title": "M5 ? truly-local profile end-to-end + first committed runlog",
        "milestone": "M5",
        "gate_type": "run_execution",
        "blocks": "M6 (Cloud Run dispatch shim builds on the truly-local profile + parity test
as its in-process control); the committed runlog DoD item (§9: truly-local must have ?1 real
sample run); the headline D13 local?cloud switch story.",
        "claude_doable_now": "MOST of M5 is doable now with zero owner involvement. The M1
scaffolding already exists (backend/config/presets.py:27 PROFILES, :33 PROFILE_TO_TARGET, :43
PROFILE_PRESETS, :104 auto-detect). Claude can write+merge: (1) the truly-local preset wiring
end-to-end (preset already in presets.py:44 ? extend to lock detector_impl/stitch-in-
process/storage=local coherently); (2) per-profile STARTUP CHECKS that fail/warn-fast (README
§4.3: 'cloud-serving without GCS creds/GPU ? loud error') ? pure config validation, unit-
testable; (3) the L4 PROFILE-PARITY test (TESTING_STRATEGY §1 L4, the load-bearing proof) ? run
one replay_csv golden trajectory through in-process truly-local vs worker+fake-gcs and assert
byte-identical windows+segment ranges; (4) L1?L3 coverage (TESTING_STRATEGY §6: L1
preset/precedence, L2 storage roundtrip, L3 stub-E2E). All deterministic, $0, no GPU. The
StorageBackend ABC + 4 backends + StubDetectorRunner replay_csv all exist (verified:
backend/storage/{base,local,s3,gcs,gdrive,ref}.py). The ONLY thing Claude cannot produce is the
committed real-GPU runlog itself.",
```

```
        "owner_prerequisite": "A real-GPU run on the owner's box (local 2070 / Linux GPU)
producing the committed truly-local runlog under runlogs/ ? rally counts + reel smoke-test
(ffmpeg -f null - exit 0 + 5-frame contact sheet) per TESTING_STRATEGY §5/L5. Owner-side because
the golden corpus + labelled videos + GPU live with the owner.",
        "unblock_value": "high",
        "recommendation": "Approve M5 as TWO PRs: a Claude-merged PR landing the truly-local
preset+startup-checks+L4 parity test+L1?L3 coverage NOW (default-safe, no behaviour change at
profile=''), then the owner does ONE real-GPU run to drop the runlog. This unblocks M6 almost
entirely on the code side.",
        "evidence": "README.md:118 (M5 row, Depends M2/M3/M4, 'owner real-GPU'); §4.3:85
(startup checks); TESTING_STRATEGY.md L4:31-34 (parity proof), L5:36-39 (owner real-GPU), §6:73
(M5 must be green); backend/config/presets.py:27,33,43,104"
      },
      {
        "id": "m6-dispatch-shim-image",
        "title": "M6 ? Cloud Run GPU dispatch shim + containerized worker image",
        "milestone": "M6",
        "gate_type": "spend",
        "blocks": "M7 (cloud-serving e2e ? the shim is what enqueues/runs the L4 GPU job);
transitively M8/M9 (dep M7) and M11/M12 (dep M7,M9).",
        "claude_doable_now": "The dispatch SHIM and the IMAGE definition are both
writable+mergeable now behind default-OFF, with mocked-dispatch tests, no GCP spend. README
§4.5:92 lists 'Cloud Run dispatch shim + container image' as net-new but the substrate exists:
the async job control plane (jobs table, claim_due_job CAS), ExecutionPolicy
WAIT_AND_RESUME/JobWaiting 'carries to Cloud Run dispatch verbatim' (§4.5; verified
backend/infrastructure/execution_policy.py). Claude can: (1) write the dispatch shim as control-
plane?job-via-storage?re-claim, exercised entirely with FAKE gcs + the stub detector
(TESTING_STRATEGY §6 M6: 'mocked-dispatch tests'); (2) author the Dockerfile
(ffmpeg+CUDA+WASB+fonts) and a CI image-build/lint check that ffmpeg/fonts/CUDA are on PATH
WITHOUT deploying it; (3) keep cloud_run target inert at profile='' / cpu-mock. The mocked-
dispatch path is exactly TESTING_STRATEGY L3's 'exercises the cloud-serving code path with no
cloud'.",
        "owner_prerequisite": "GCP spend approval to actually push the image to Artifact
Registry and run it on Cloud Run+GPU (L4) ? the image-builds-and-runs-on-real-hardware
validation. ADR-013 = GCP sole compute stack; D7 = scale-to-zero (no warm pool). Owner must
greenlight the (small, scale-to-zero) GCP spend before any real dispatch.",
        "unblock_value": "high",
        "recommendation": "Approve M6 code NOW as a default-OFF PR (shim + Dockerfile + mocked-
dispatch tests + image-build CI), and separately pre-authorize a bounded scale-to-zero GCP spend
so M7 can run immediately after. The shim/image is the single biggest chunk of independent code
that the cloud track needs.",
        "evidence": "README.md:119 (M6 row, Depends M5, '? owner (GCP spend)'); §4.5:92 (net-new
shim+image; WAIT_AND_RESUME carries verbatim); §4.4:89 (claim_due_job supports queued+sync);
D7:34 (scale-to-zero); TESTING_STRATEGY.md §6:74 (M6 mocked-dispatch + image builds)"
      },
      {
        "id": "m7-cloud-serving-e2e",
        "title": "M7 ? cloud-serving profile e2e on L4 + DEPLOY_REPORT",
        "milestone": "M7",
        "gate_type": "run_execution",
        "blocks": "M8, M9 (both dep M7), and transitively M11, M12 (dep M7,M9); the §9 DoD cloud
DEPLOY_REPORT item.",
        "claude_doable_now": "The cloud-serving CODE PATH is largely landed by M2 (input_backend
URI-aware: upload<2GB + gdrive/bucket fetched to scratch, README changelog:159) + M6
(dispatch+image). What Claude can still write now: (1) the signed-URL reel delivery wiring
(StorageBackend already has gcs.py) under fake-gcs tests; (2) the cloud-serving preset's
skip_ai_handoff=False / Gemini-ON default per D11 (quality floor) ? config only; (3) the
DEPLOY_REPORT TEMPLATE + the contact-sheet/ffmpeg-smoke harness script that the owner will run.
The actual e2e on a real L4 (real detection bytes, real reel) CANNOT be produced locally ? no
GPU/cloud here.",
        "owner_prerequisite": "Owner runs the real cloud-serving e2e on a GCP L4 (upload<2GB +
gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL) and commits the DEPLOY_REPORT
(real sample runs + contact sheet) ? GCP spend + owner-side golden corpus. This is the §9 DoD
gate and is real-money + real-hardware.",
        "unblock_value": "high",
```

```
        "recommendation": "Treat M7 as owner-run after M6 lands. Claude pre-stages everything
runnable ($0): signed-URL delivery, Gemini-ON cloud preset, the DEPLOY_REPORT template + smoke-
test/contact-sheet script ? so the owner's run is one-shot. Spend authorization can be batched
with M6's.",
        "evidence": "README.md:120 (M7 row, Depends M6, '? owner (spend)'); §4.2:76 (cloud-
serving orchestration); D11:38 (Gemini-ON floor); changelog:159 (M2 already does
input<2GB+gdrive/bucket); TESTING_STRATEGY.md §6:75 (M7 = owner L5 cloud DEPLOY_REPORT),
§5:57-62 (runlog contents)"
      },
      {
        "id": "m8-cost-ledger-pricebook",
        "title": "M8 ? per-job cost ledger + pricebook + cost API",
        "milestone": "M8",
        "gate_type": "spend",
        "blocks": "M10 (byo_worker design dep M8,M9); the §9 DoD 'cost ledger makes
GPU+stitch+egress cost visible per job' item.",
        "claude_doable_now": "NEARLY ALL of M8 is Claude-doable now with mocked/seeded data, no
real billing. D8 LOCKED the design (pricebook = versioned DB table, USD-internal/INR-display at
configurable FX). Claude can write+merge: (1) the v_ DB migration adding
owner_id/gpu_active_seconds/wall_seconds/bytes_out/cost_usd/cost_breakdown_json (the migration
ladder is PRAGMA user_version in backend/infrastructure/database.py ? verified, v4 already added
user_id columns at :227); (2) the versioned pricebook TABLE + seeding mechanism; (3) the USD
compute + INR display at a configurable FX rate; (4) the aggregation + /api/.../cost endpoint;
(5) unit tests on pricebook+aggregation (TESTING_STRATEGY §6 M8). All pure logic, deterministic,
$0. The metering hooks (gpu_active_seconds/wall_seconds/bytes_out) can be captured in the M6/M7
job path.",
        "owner_prerequisite": "REAL GCP prices to seed the first pricebook version (D8: 're-
versioned when GCP prices change'; matches GCP billing as one source of truth) AND a reconcile
probe of ledger numbers vs actual platform billing on a real run (TESTING_STRATEGY §6 M8:
'reconcile probes vs platform billing in the runlog'). The owner must supply real per-unit
prices + an FX rate, and confirm the ledger reconciles against a real GCP invoice line.",
        "unblock_value": "high",
        "recommendation": "Approve M8 schema+pricebook+aggregation+API code NOW with a
PLACEHOLDER/seeded pricebook version (clearly marked unverified). Owner only needs to (a) hand
over the real GCP unit prices + FX, and (b) reconcile on the first M7 run. The entire code
surface lands independently.",
        "evidence": "README.md:121 (M8 row, Depends M7, '? owner (billing)'); D8:35 (versioned
DB table, USD/INR, re-version on price change); backend/infrastructure/database.py:221-236
(user_version migration ladder, v4 user_id precedent); TESTING_STRATEGY.md §6:76 (M8 unit tests
+ reconcile probes)"
      },
      {
        "id": "m9-edge-auth-consent",
        "title": "M9 ? edge auth (Cf-Access) + per-tenant scoping + DPA/consent gate",
        "milestone": "M9",
        "gate_type": "counsel_consent",
        "blocks": "M10 (dep M8,M9); M11 (dep M7,M9 ? the data-flywheel consent gate); M12 (dep
M7,M9 ? OAuth Drive needs auth); the §9 DoD 'edge auth gates multi-tenant' item; public non-
owner serving.",
        "claude_doable_now": "The auth PLUMBING is Claude-doable now; the CONSENT/ToS content is
not. D9 LOCKED the design (Cloudflare Access; verify Cf-Access JWT signature; lock Cloud Run
ingress to the CF proxy). Claude can write+merge behind default-OFF: (1) Cf-Access header
EXTRACTION + JWT-signature verification (so a direct Cloud Run hit can't spoof the identity
header ? D9's explicit anti-spoof requirement), with a header-trust/spoofing-guard test
(TESTING_STRATEGY §6 M9); (2) per-tenant scoping on user_id ? the v4 columns + partial indexes
already EXIST (database.py:227-236, verified), so wiring queries to filter on user_id is
independent code; (3) per-tenant scoping tests. What Claude CANNOT do: produce the counsel-
reviewed DPA/ToS text or decide the consent UX ? D10 says 'ToS counsel-reviewed', 'live
training-on-uploads gated to M9 (public signup)'.",
        "owner_prerequisite": "Counsel-reviewed ToS/DPA for non-owner uploads + the consent-
capture wording/flow (D10: adults-only attestation, derived-data + auto-delete-raw, verifiable
consent), AND the real Cloudflare Access tenant config (CF app + the JWT issuer/audience to
verify against). These are legal-review + real-credential items only the owner/counsel can
supply.",
        "unblock_value": "high",
```

```
        "recommendation": "Approve M9 auth-plumbing + per-tenant-scoping + spoof-guard tests as a default-OFF Claude PR NOW (config-gated, inert until a CF issuer is set). Keep the ToS/DPA text + consent UX strictly owner/counsel-gated. This unblocks the structural prerequisite that M11 and M12 both depend on, without waiting on legal.",
        "evidence": "README.md:122 (M9 row, Depends M7, '? owner (privacy/counsel)'); D9:36 (Cf-Access, verify JWT, lock ingress); D10:37 (consent/ToS counsel-reviewed, gated to M9); database.py:221-236 (v4 user_id columns + partial indexes exist); TESTING_STRATEGY.md §6:77 (M9 per-tenant + spoofing-guard tests)"
      },
      {
        "id": "m10-byo-worker",
        "title": "M10 ? byo_worker / zero-infra-BYO (design-only, DEFERRED)",
        "milestone": "M10",
        "gate_type": "housekeeping",
        "blocks": "Nothing downstream in this track ? it is the terminal, explicitly-deferred milestone (enum/seam reserved so it isn't precluded).",
        "claude_doable_now": "Almost nothing should be built now ? M10 is explicitly DESIGN-ONLY and DEFERRED (README §4.2:78 'byo_worker (deferred) ... enum/seam reserved'; D3:30 'byo_worker reserved/deferred'). The only safe Claude work: keep the compute_target enum + job-table-scoping seam open enough that BYO 'slots into the enum without core changes' (ADR-014 consequence :711) ? but that reservation is already done in M1. No new code is warranted; a premature BYO build would violate the deferral.",
        "owner_prerequisite": "Explicit owner approval to UN-defer it at all (README:123 '? explicit owner approval'). BYO carries 'a real ops/support burden (deferred, power-user only)' per ADR-014 consequence :712 ? a product/strategy decision, not a code gate.",
        "unblock_value": "low",
        "recommendation": "Leave M10 DEFERRED. Do NOT spend code budget here. Revisit only on an explicit owner decision to serve privacy-sensitive enterprise tenants on their own GPU; until then the reserved enum/seam (already in M1) is sufficient.",
        "evidence": "README.md:123 (M10 row, Depends M8/M9, 'design-only DEFERRED', '? explicit owner approval'); §4.2:78 (byo_worker deferred, enum/seam reserved); D3:30; ADR-014 consequences DECISIONS.md:711-712 (slots into enum; real ops burden, deferred)"
      },
      {
        "id": "m11-data-flywheel",
        "title": "M11 ? data-flywheel harness (capture?store?shadow-eval?goldenize)",
        "milestone": "M11",
        "gate_type": "counsel_consent",
        "blocks": "The path to flipping Gemini OFF (D11 quality floor) ? the §9 DoD 'owned model on a measured path to the D11 floor' item; the paper-coauthor flywheel aim.",
        "claude_doable_now": "The HARNESS MECHANICS are Claude-doable now on synthetic/fixture data; the live capture of REAL consented user footage is not. D12 put this fully IN SCOPE and it 'reuses data_pipeline/ + backend/eval/ + the M9 consent gate'. Claude can write+merge behind default-OFF: (1) the capture path (source upload + Gemini reel + rally output ? workspaces/<video_id>/) as the consented branch vs the no-consent auto-delete branch ? wired but inert until M9 consent flips on; (2) the durable-store step into the consented-retention bucket (StorageBackend gcs.py exists); (3) the shadow-eval step calling backend/eval/experiment.compare (the dual-eval-set harness already exists per [[eval-dual-set-regression]]) on FIXTURE videos ? owned-vs-Gemini gap tracking; (4) the promote-to-golden-TRAINING-row hook into data_pipeline/. All exercisable with golden fixtures, $0, no real user data. CANNOT do: ingest real consented adult footage (needs M9 consent live + real uploads).",
        "owner_prerequisite": "Depends on M9's consent gate being LIVE (D10: adults-only attestation, verifiable consent, counsel-reviewed ToS, raw-retention gated by consent) ? i.e. the same counsel/consent prerequisite as M9, plus the human-review labeling step to actually promote captured videos to the golden TRAINING set. Owner/counsel must clear consented retention of real footage before any live capture runs.",
        "unblock_value": "medium",
        "recommendation": "Approve M11 harness mechanics as a default-OFF Claude PR NOW (capture/store/shadow-eval/promote wired against fixtures + the existing experiment.compare/data_pipeline seams), gated behind the M9 consent flag so it stays inert until consent is live. Live capture waits on the M9 counsel/consent prerequisite ? no separate new gate.",
        "evidence": "README.md:124 (M11 row, Depends M7/M9, '? owner (consent/privacy)'); §4.6:94-100 (the 4-step harness, reuses data_pipeline/+backend/eval/+M9 gate); D12:39 (in-scope, ties consent+eval harness); D10:37 (adults-only, consent gating)"
```

```
      },
      {
        "id": "m12-gdrive-oauth-backend",
        "title": "M12 ? gdrive-api (OAuth) StorageBackend (Northstar Drive round-trip)",
        "milestone": "M12",
        "gate_type": "creds_calibration",
        "blocks": "The D14 NORTHSTAR (point-at-Drive ? reel back in Drive next to the source);
the mobile/cloud-native app-like flow (khelsutra UI track depends on this engine capability).",
        "claude_doable_now": "The BACKEND CODE is Claude-doable now against a mocked Drive API;
the OAuth app + real consent flow are not. D14 resolved §8#7 ? cloud needs a gdrive-api (OAuth)
StorageBackend 'distinct from the local mount backend; slots into the StorageBackend ABC'. The
ABC + a local gdrive MOUNT backend already exist (backend/storage/gdrive.py, base.py ?
verified). Claude can write+merge behind default-OFF: (1) a new gdrive_api StorageBackend
implementing the ABC (put/exists/stat/list/fetch/delete) against an INJECTED/FAKE Drive client ?
read source + write reel back next to source ? exactly the L2 storage-roundtrip pattern
(TESTING_STRATEGY §1 L2: 'injected fake ... gdrive=tmp_path mount'); (2) the StorageRef URI
scheme for it; (3) roundtrip tests with the fake client. CANNOT do: register a real Google Cloud
OAuth app, obtain client creds, or run the real per-user consent flow.",
        "owner_prerequisite": "A registered Google OAuth app (client ID/secret, scopes for Drive
read+write), the per-user OAuth consent screen, and the verification/privacy posture Google
requires for Drive write scopes (D14: 'per-user OAuth consent ... write reel back'). These are
real-credential + Google-app-registration + consent items only the owner can stand up.",
        "unblock_value": "medium",
        "recommendation": "Approve M12 backend code as a default-OFF Claude PR NOW (gdrive_api
StorageBackend + URI scheme + fake-client roundtrip tests against the existing ABC), so the
engine 'does not preclude' the Northstar (D14). The real OAuth app + per-user consent flow is
the owner prerequisite and can follow once M9 auth is live.",
        "evidence": "README.md:125 (M12 row, Depends M7/M9, '? owner (OAuth/consent)'); D14:41
(gdrive-api OAuth backend, read source + write reel back, slots into StorageBackend ABC);
§8#7:139 (RESOLVED?D14); backend/storage/{base,gdrive,ref}.py (ABC + mount backend exist);
TESTING_STRATEGY.md L2:22 (storage roundtrip with injected fakes)"
      },
      {
        "id": "q2-stitch-placement",
        "title": "§8 Q2 ? stitch placement: co-located in detect job vs split to CPU service",
        "milestone": "M6/M7 (design input)",
        "gate_type": "design_choice",
        "blocks": "M6/M7 cloud-serving job structure ? whether stitch runs inside the GPU job or
a separate CPU Cloud Run service.",
        "claude_doable_now": "Claude builds to the documented DEFAULT now: co-located stitch in
the detect job (§8#2: 'Default co-located unless metrics say otherwise'; D4:31 + §4.4:89 already
lock stitch-on-Cloud-Run co-located for metering). The job table + claim_due_job 'already
supports both' (§4.4:89), so a future split is non-breaking. No new code is blocked ? the
default is already the design.",
        "owner_prerequisite": "A cheap owner CONFIRM that co-located stays the v1 default (it
already is per D4). The only thing that would change it is real bursty-compile METRICS from M7
runs showing stitch should split to a CPU service ? i.e. data the owner gathers post-M7, not a
v1 blocker.",
        "unblock_value": "low",
        "recommendation": "Confirm co-located (the documented default). Build M6/M7 with stitch
in the GPU job; revisit a split only if M7 metrics show compiles get bursty. This is effectively
already decided by D4 ? just needs a one-word owner OK.",
        "evidence": "README.md §8#2:138 ('Default co-located unless metrics say otherwise');
D4:31 (stitch on Cloud Run, metered); §4.4:89 (claim_due_job supports both co-located + queued
CPU-only fallback)"
      },
      {
        "id": "q8-nvdec-nvenc-vs-libx264",
        "title": "§8 Q8 ? NVDEC/NVENC in the Cloud Run image vs libx264",
        "milestone": "M6/M7 (image design)",
        "gate_type": "design_choice",
        "blocks": "M6 container image build (which encoder/decoder to bake in) and the M7 stitch
wall-time/cost; touches the parity guarantee.",
        "claude_doable_now": "Claude can build the M6 image with the PARITY-SAFE default now:
libx264 for STITCH (the core STITCH contract is 'per-clip libx264 trim ? concat', §4.1:68 ?
```

```
byte-deterministic, which D5/parity depend on). NVDEC for DECODE is a separable speed lever
(ADR-012 named NVDEC). Claude can structure the image so the encoder is a config knob,
defaulting libx264 (determinism), without blocking. The tension is explicit in Q8: NVENC cuts
cost but threatens cross-run determinism/parity.",
        "owner_prerequisite": "An owner design pick: accept NVENC's cost/wall-time win at the
risk of breaking stitch byte-determinism (the parity guarantee D5 + the L4 parity test rely on
libx264 determinism), OR keep libx264 for parity. This is a quality/cost tradeoff judgment the
owner owns; ideally informed by an M7 wall-time/cost measurement.",
        "unblock_value": "low",
        "recommendation": "Default libx264 for the stitch ENCODE (protects the parity guarantee
+ the L4 test), and optionally enable NVDEC for DECODE only (speed without touching output
bytes). Defer NVENC encode unless M7 cost data justifies relaxing parity ? make it a config knob
so the pick is reversible.",
        "evidence": "README.md §8#8:140 (NVDEC/NVENC vs libx264 determinism/parity); §4.1:68
(STITCH = per-clip libx264 trim, deterministic); D5:32 (parity at window+stitch); ADR-012
lineage table:54 (NVDEC named as decode lever); TESTING_STRATEGY.md §7:82 (don't let determinism
slip)"
      },
      {
        "id": "q9-input-cap-2gb-vs-4gb",
        "title": "§8 Q9 ? input cap: <2GB hard reject vs accept to 4GB upload cap",
        "milestone": "M2/M7 (input routing)",
        "gate_type": "design_choice",
        "blocks": "The cloud-serving input-routing UX (client-side reject threshold vs server
route-larger-elsewhere); minor M7 polish.",
        "claude_doable_now": "Claude can implement EITHER policy now ? the chunked-upload router
'(<2GB)' already exists (§4.5:91-92, and M2 landed input<2GB per changelog:159). A hard <2GB
client reject vs accept-to-4GB-then-route-larger is a small config/validation change Claude can
write behind a config knob with both branches tested. Not structurally blocking.",
        "owner_prerequisite": "A product UX pick: is <2GB a hard client reject, or do we accept
to the 4GB upload cap and route larger files elsewhere (§8#9)? This is the owner's call on what
the hosted service promises users ? cheap to decide, no spend/creds.",
        "unblock_value": "low",
        "recommendation": "Pick hard-reject-at-<2GB for v1 simplicity (matches the already-
shipped M2 chunked-upload threshold), with a clear client-side error; defer 'route larger files
elsewhere' to a later milestone. Implement as a config cap so raising it to 4GB later is a one-
line change.",
        "evidence": "README.md §8#9:141 (<2GB hard reject vs 4GB cap); §4.5:91-92 (chunked-
upload router <2GB exists); changelog:159 (M2 landed input<2GB); README §1:20 ('small local
uploads <2GB')"
      },
      {
        "id": "q10-dir-name",
        "title": "§8 Q10 ? directory name COMPUTE_DECOUPLED_SERVING/ vs CLOUD_RUN_GPU_SERVING/",
        "milestone": "housekeeping",
        "gate_type": "design_choice",
        "blocks": "Nothing functional ? purely the project-doc directory name; all docs/anchors
already use COMPUTE_DECOUPLED_SERVING/.",
        "claude_doable_now": "Nothing to build ? the chosen name COMPUTE_DECOUPLED_SERVING/ is
already in use across the whole doc tree (the project dir, all §10 anchors, ADR-014's 'Full
design: docs/COMPUTE_DECOUPLED_SERVING/'). Claude would only need to confirm-or-rename, which is
a trivial git mv if the owner prefers the old name.",
        "owner_prerequisite": "A one-word owner CONFIRM that COMPUTE_DECOUPLED_SERVING/ (already
chosen, §7 'chosen') is final and CLOUD_RUN_GPU_SERVING/ is dropped. The doc already treats it
as chosen; this just closes the open marker.",
        "unblock_value": "low",
        "recommendation": "Confirm COMPUTE_DECOUPLED_SERVING/ as final (it is already the name
everywhere and ADR-014 references it). Close §8#10. No code/doc change needed beyond marking it
resolved.",
        "evidence": "README.md §8#10:142 ('COMPUTE_DECOUPLED_SERVING/ (chosen) vs keep
CLOUD_RUN_GPU_SERVING/ ? confirm'); §7 naming note:7; ADR-014 DECISIONS.md:688 ('Full design:
docs/COMPUTE_DECOUPLED_SERVING/')"
      }
    ]
  },
```

```
  {
    "area": "Cloud-serving compute substrate (M6 Cloud Run dispatch shim + container image; M7
L4 e2e deploy) ? backend/worker, backend/api job control plane,
backend/infrastructure/database.py jobs table, backend/storage,
trajectory_hybrid._resolve_runner",
    "gates": [
      {
        "id": "m6-cloud-run-dispatch-shim",
        "title": "M6 ? Cloud Run GPU dispatch shim + containerized worker image (mocked-dispatch
tests now; spend deferred)",
        "milestone": "M6",
        "gate_type": "design_choice",
        "blocks": "M7 (L4 e2e), and the whole cloud-serving profile being runnable. But it does
NOT block writing the code: the shim + Dockerfile are net-new files that ship default-OFF (only
active when compute_target=cloud_run, which today is inert) and are fully mockable.",
        "claude_doable_now": "SUBSTANTIAL. The entire dispatch path can be written + merged NOW
behind default-OFF with mocked-dispatch tests, no GCP. Concretely: (1) NEW backend/compute/
registry mirroring backend/storage (a ComputeBackend ABC + a LocalCompute that runs in-process =
today's behaviour byte-identical, + a CloudRunCompute that, given a job, stages inputs to gs://
via the existing storage.put and triggers a Cloud Run Job via an INJECTABLE client ? same DI
pattern GCSStorage already uses, gcs.py:18-25 takes client=None for tests). (2) Wire the
selector off deployment.compute_target (already an enum incl 'cloud_run', models.py:351; cloud-
serving preset already sets it, presets.py:51-54) ? default '' / 'local_gpu' keeps the in-
process path, so zero core-caller change. (3) Re-use the control plane verbatim: on dispatch,
the control-plane job raises JobWaiting (job_worker.py:41-56) to park itself (set_job_waiting,
database.py:559-569) and re-claim via claim_due_job (database.py:588-608) when the Cloud Run
job's output appears in the bucket ? the doc itself says WAIT_AND_RESUME 'carries to Cloud Run
dispatch verbatim' (README §4.5). (4) The container entrypoint already EXISTS: `python -m
backend.worker infer --video-uri gs://? --out-uri gs://?` (worker/__main__.py:91-176)
pulls?runs?pushes via storage.fetch/put; no worker rewrite. (5) NEW Dockerfile baking ffmpeg +
CUDA + the WASB-SBDT repo clone + weights + fonts ? the exact external deps
NativeWasbRunner.healthcheck asserts (weights_path, repo src/, python_bin w/ torch+cuda;
native_wasb_runner.py:157-191). The Dockerfile can be lint/hadolint-checked + a build-arg matrix
unit-tested without a GPU; a CI smoke can run the image's cpu_mock/stub path
(detector_impl=stub, skip_ai_handoff=true) which is GPU-free by design (__main__.py:9-11).
Tests: fake Cloud Run client asserts the correct job-create call + bucket staging; a
JobWaiting?re-claim round-trip on an in-memory sqlite jobs table asserts park/resume; profile-
parity test (cpu-mock vs local) stays green. All $0, all offline.",
        "owner_prerequisite": "A design pick + a go-ahead to WRITE the net-new code now (the
shim contract: trigger Cloud Run Jobs vs Cloud Run Services-with-GPU; stitch co-located in the
detect job per D4/§8 Q2; whether the control plane polls the bucket for completion vs the job
calls back). The owner need NOT provision anything or spend for the CODE to land ? only to RUN
it (that's M7). Secondary: confirm §8 Q8 (NVENC vs libx264 in the image) and Q10 dir name, but
those don't block the shim skeleton.",
        "unblock_value": "high",
        "recommendation": "APPROVE writing M6 now + defer spend. One PR: a backend/compute
ComputeBackend registry (LocalCompute=identity default-ON, CloudRunCompute behind
compute_target=cloud_run, default-OFF) + an injectable Cloud Run client + a Dockerfile
(entrypoint=`python -m backend.worker infer`), with fully-mocked dispatch tests + a stub/cpu-
mock image smoke. Mirror the storage-backend DI exactly so it's testable with no cloud. The real
L4 invocation is the only thing held for M7. Recommend Cloud Run *Jobs* (not Services) for the
GPU detect+stitch unit (matches the async upload?poll?reel shape + scale-to-zero D7), and keep
stitch co-located (D4).",
        "evidence": "compute_target enum incl 'cloud_run' models.py:351; cloud-serving preset
presets.py:51-54; jobs table database.py:187-204; claim_due_job CAS database.py:588-608;
set_job_waiting/seconds_until_next_due database.py:559-586; JobWaiting + drain/wake
job_worker.py:41-152; worker container entrypoint cmd_infer worker/__main__.py:91-176;
storage.fetch/put + GCS download/upload/signed_url backend/storage/__init__.py:106-119 +
gcs.py:30-63; NativeWasbRunner external deps healthcheck native_wasb_runner.py:157-191;
_resolve_runner seam trajectory_hybrid.py:66-87; README §4.5 'WAIT_AND_RESUME carries to Cloud
Run dispatch verbatim' + net-new list 'Cloud Run dispatch shim + container image'; NO
backend/compute dir, NO Dockerfile, deploy/gcp = VM-only (provision.sh has no `gcloud run`)."
      },
      {
        "id": "m7-cloud-serving-l4-e2e",
```

```
        "title": "M7 ? cloud-serving e2e on a real L4 (upload<2GB / gdrive / bucket ?
detect+stitch on Cloud Run ? reel via signed URL) + DEPLOY_REPORT",
        "milestone": "M7",
        "gate_type": "run_execution",
        "blocks": "The committed cloud-serving runlog/DEPLOY_REPORT (DoD §9), and everything
downstream that needs a proven hosted path: M8 cost ledger (needs real gpu_active/wall/egress
numbers), M9 edge auth on a live endpoint, M11 flywheel capture, M12 Drive round-trip. Also the
only thing that proves real-GPU detection quality (README §6: green CI proves plumbing, NOT
field quality).",
        "claude_doable_now": "ALMOST NONE that is new beyond M6. M7 is a RUN, not a code
milestone. Everything writable is already in M6 (the shim, the image) or earlier (chunked upload
uploads.py handles <2GB, signed-URL reel delivery exists via GCSStorage.signed_url gcs.py:61-63,
gdrive mount backend exists). I CAN pre-write the M7 scaffolding offline: a DEPLOY_REPORT.md
template + a sample-run harness script that drives upload?poll?signed-URL against a FAKE/mock
dispatch (proving the client-side e2e shape, contact-sheet generation, report layout) so the
only missing piece on the day is pointing it at the real Cloud Run URL. But the load-bearing
artifact ? a real L4 detect+stitch run with real numbers + a contact sheet ? is owner-only.",
        "owner_prerequisite": "GCP SPEND + a real run: deploy the M6 image to Cloud Run with an
L4 GPU, run a sample video (or a few) end-to-end on the real substrate, and capture the
DEPLOY_REPORT. This needs billing enabled, the GPU quota (deploy/gcp/README notes asia-south1 L4
is often stocked-out ? may need zone sweep / Singapore fallback), and the owner's
golden/labelled corpus (the machine-side check explicitly handed to the owner per CLAUDE.md). No
amount of mocking substitutes for the cross-GPU parity + real-detection-quality observation (D5,
README §6).",
        "unblock_value": "low",
        "recommendation": "HOLD for owner spend ? but pre-stage everything around it. Land M6
(mocked) first; then I write the DEPLOY_REPORT template + a mock-driven sample-run harness so
the owner's real L4 run is a single deploy+invoke that fills in numbers. Sequence the owner's
first spend as the smallest viable run (one short clip, T4/L4 cheapest-adequate, DELETE the
service after per the gcp-vm-always-delete rule) to validate the path before any wider run. Do
NOT attempt to fake this milestone green ? its whole value is the real-substrate observation.",
        "evidence": "M7 row gated '? owner (spend)' README §7; DoD requires a committed cloud-
serving DEPLOY_REPORT §9; chunked upload <2GB uploads.py:64-145 (max_upload_mb 4096, part cap);
signed-URL reel delivery GCSStorage.signed_url gcs.py:61-63; gdrive mount backend
get_storage('gdrive') backend/storage/__init__.py:88-90; README §6 'green CI proves plumbing?
does NOT prove field quality nor real-GPU detection'; D5 cross-GPU sub-pixel parity (parity
enforced at WINDOW+STITCH); deploy/gcp/README capacity note (asia-south1 GPUs frequently
stocked-out); CLAUDE.md 'hand the heaviest machine-side checks (real video, full GPU suite) to
the owner'."
      }
    ]
  },
  {
    "area": "code-reality for M8 cost ledger, M9 edge-auth (split auth vs consent), M12 gdrive-
api OAuth ? pure-code-now vs owner creds/calibration/counsel",
    "gates": [
      {
        "id": "m8-cost-ledger",
        "title": "M8 per-job cost ledger: v6 migration + pricebook table + aggregation +
/api/.../cost ? codeable NOW with placeholder/zero prices",
        "milestone": "M8",
        "gate_type": "creds_calibration",
        "blocks": "Real per-video/per-run billing figures (and any resale margin) shown to non-
owner users. The PLUMBING is not blocked; only the NUMBERS being TRUE is blocked.",
        "claude_doable_now": "Almost the entire mechanism, default-OFF, with mocked tests. (1)
v6 migration on the existing ladder (database.py:91-106 + _add_column_idempotent
database.py:61-77): ALTER jobs ADD COLUMN owner_id/compute_backend/gpu_type/gpu_active_seconds/w
all_seconds/bytes_out/gemini_cost_usd/pricebook_version/cost_usd/cost_breakdown_json + CREATE
TABLE pricebook(version,resource,gpu_type,unit_price_usd,currency,effective_at) +
training_contributions, all additive/NULLable so NULL=local byte-identical (mirrors the v4
user_id pattern, database.py:219-237); bump PRAGMA user_version=6; a test asserts the new
version (drift fails CI per database.py:86-88). (2) Writer/reader methods (set_job_cost,
get_pricebook(version), upsert_pricebook, attribute_training_run) following the _execute_write
swallow+log convention (database.py:44-59). (3) A pure cost_usd(job)=? usage×rate(version)
calculator + USD->INR display formatting (D8, README §2 D8) as a standalone module with a SEEDED
```

placeholder/zero pricebook row (version='placeholder-v0', all unit_price_usd=0.0) ? fully unit-testable with synthetic usage. (4) gpu_active_seconds/wall_seconds capture: wrap the worker GPU/stitch section with a timestamp recorder in job_worker._run_claimed_job (job_worker.py:72-97) ? pure timing, no GPU needed to test. (5) GET /api/jobs/{id}/cost + GET /api/videos/{video_id}/cost endpoints (mirroring get_job main.py:303-322), scoped by owner_id, returning cost_breakdown_json+pricebook_version. All default-OFF: with the zero/placeholder pricebook the cost is $0 and nothing user-facing changes.",
        "owner_prerequisite": "REAL CALIBRATED PRICES, not code: (a) the actual GCP L4/T4 $/GPU-second + egress $/GB + storage GB-hr rates to seed a real pricebook version (the worker-measured-rate path 05-COST-ATTRIBUTION.md:30, since Cloud Run isn't live until M7); (b) the FX rate + any resale `margin` policy for INR display (D8); (c) sign-off to FLIP the ledger from $0-placeholder to live billing (a spend/billing decision). Optionally a real GCP invoice for the monthly reconciliation_factor (05-COST-ATTRIBUTION.md:94). Also note gemini_cost_usd has NO real source yet ? gemini.py metrics is timing-only (gemini.py:43-47, returns segments,metrics gemini.py:146), so token-cost capture is either a separate small code add or stays a placeholder until the owner supplies a per-call estimate.",
        "unblock_value": "high",
        "recommendation": "Approve building the full ledger NOW behind a seeded zero/placeholder pricebook (version='placeholder-v0'), default-OFF and $0-valued, with mocked tests. This unblocks the entire schema+aggregation+endpoint surface independently; the owner's ONLY later action is a one-row pricebook INSERT with real rates + an FX/margin pick + the flip-to-live OK. Recommend the worker-measured gpu_active_seconds path (05-COST-ATTRIBUTION.md:30) so it works pre-Cloud-Run, and keep gemini_cost_usd a nullable placeholder until M7.",
        "evidence": "database.py:188-203 (jobs table, no cost cols), database.py:219-237 (v4 user_id additive-NULL pattern to mirror), database.py:61-77 (_add_column_idempotent), database.py:86-106 (version ladder, asserted in tests), database.py:44-59 (_execute_write convention); docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md:7,36-61,68-78 (schema delta + owner_id), :30 (worker-measured rate), :94 (reconciliation); README §2 D8 (line 35); job_worker.py:72-97 (where to time); main.py:303-322 (endpoint pattern); gemini.py:43-47,146 (timing-only metrics, no token cost)"
      },
      {
        "id": "m9-auth",
        "title": "M9 edge-auth (codeable half): Cf-Access JWT signature verification + per-tenant scoping on user_id ? fully writable + unit-testable NOW with a fake JWKS",
        "milestone": "M9",
        "gate_type": "creds_calibration",
        "blocks": "Multi-tenant request identity actually being trusted in production (a real CF team-domain protecting real Cloud Run). The verification LOGIC + scoping is not blocked; only the real CF config values are.",
        "claude_doable_now": "The whole auth-verification + scoping half, default-OFF, with mocked crypto. (1) Add PyJWT[crypto] (or python-jose) ? NO such dep exists today (requirements.txt/pyproject.toml have no jwt/cryptography). (2) A FastAPI dependency/middleware that extracts the Cf-Access JWT from the `Cf-Access-Jwt-Assertion` header, fetches+caches the team JWKS, verifies signature+exp+iss+aud, and maps the verified identity (email/sub) -> user_id. Unit-test it END-TO-END offline by generating an RSA keypair in the test, signing a token, serving a FAKE JWKS (no network), and asserting accept/reject (bad sig, expired, wrong aud, missing header). (3) Per-tenant scoping: thread the resolved user_id into create_job/get_job and the video/candidate queries ? the v4 NULLable user_id columns + partial indexes ALREADY EXIST (database.py:227-237) and the v5 user_models store is keyed on it (database.py:256-285); jobs still needs a user_id/owner_id column (folds into the M8 v6 migration). (4) Default-OFF gate: a config flag (e.g. security.edge_auth_enabled=False) so when off the API behaves byte-identically to today (NULL user_id = local, every existing query unaffected ? database.py:227-237 invariant). (5) Tighten the CORS posture the code already flagged as a placeholder (main.py:40-41).",
        "owner_prerequisite": "Real Cloudflare config VALUES + infra, not code: (a) the CF Access team domain / JWKS issuer URL, (b) the application AUD tag (CF Access app audience), (c) locking Cloud Run ingress to the CF proxy so the header can't be spoofed by a direct hit (D9, README line 36 ? an infra/deploy step, owner-side, needs M7 live). These are configuration+deployment the owner provisions in their CF dashboard; none is needed to write or unit-test the verifier.",
        "unblock_value": "high",
        "recommendation": "Approve writing the full Cf-Access JWT verifier + user_id scoping NOW, default-OFF behind security.edge_auth_enabled, unit-tested with an in-test RSA keypair + fake JWKS (zero network, zero CF account). Owner later supplies only team-domain+AUD config

strings and the ingress-lock deploy step. This is high-value: it makes the v4/v5 user_id columns finally load-bearing without touching the local default path.",
        "evidence": "main.py:40-48 (CORS wide-open, allow_credentials=False, comment 'unsafe to carry into the auth/tenancy work'); grep found NO Cf-Access/JWT/JWKS/Authorization handling anywhere except the wide-open CORS in main.py; requirements.txt/pyproject.toml have NO PyJWT/cryptography/jose; database.py:227-237 (v4 NULLable user_id + partial indexes, NULL=local invariant), database.py:256-285 (v5 user_models keyed on user_id); README §2 D9 (line 36, verify Cf-Access JWT + lock ingress), §4.5 'edge-auth (v4 user_id columns exist)' line 92; main.py:292-300 (create_job ? where to inject user_id)"
      },
      {
        "id": "m9-consent",
        "title": "M9 DPA/consent gate (the COUNSEL half): ToS/DPA review + adults-only attestation + derived-data/auto-delete policy ? NOT codeable to completion without owner+counsel",
        "milestone": "M9",
        "gate_type": "counsel_consent",
        "blocks": "Any live training-on-uploads from non-owner users (the D10 free-tier data-for-reels trade) and therefore the M11 data-flywheel's raw-retention path. Public signup with the train-on-uploads bargain cannot ship until this clears.",
        "claude_doable_now": "Only the MECHANISM around a consent decision, default-OFF, with the policy values left blank/conservative: (1) a NULLable consent_status + is_adult_attested + retention_class column set on the job/video row (folds into the M8/M9 v6 migration), defaulting to the NO-CONSENT path (reel yes, training pool NO, raw auto-deleted) which is the safe default that needs no counsel. (2) The capture-vs-auto-delete BRANCH in the worker keyed on that flag (so a consented adult job CAN be captured to the workspace per M11/D12, but the flag is never set True by default). (3) An attestation field plumbed from the API request. What I CANNOT do: write the actual ToS/DPA text, decide what counts as valid adult attestation / verifiable parental consent (DPDP/GDPR Art.9), or approve enabling the training-retention path ? those are legal judgments.",
        "owner_prerequisite": "A counsel-reviewed ToS/DPA and an explicit policy decision (D10, README line 37): (a) counsel sign-off on the data-for-reels bargain + privacy posture; (b) the precise adults-only attestation + minor-exclusion rule that is legally sufficient; (c) the derived-data + raw-auto-delete retention policy text; (d) approval to FLIP the consent gate from default-OFF (no-train) to live train-on-uploads. This is a legal/consent gate, not a code gate ? explicitly counsel-gated to M9 (D10) and owner-gated per CLAUDE.md (paid-LLM/training guardrails + minors exclusion).",
        "unblock_value": "low",
        "recommendation": "Keep this gate OWNER+COUNSEL and do NOT build toward enabling it. The only safe Claude work is the default-OFF no-consent mechanism (consent flag column + capture-vs-delete branch that always defaults to delete/no-train), which can ride the M9 v6 migration. Recommend the owner schedule counsel review in parallel with M9-auth (which is independent and high-value); the auth half should not wait on the consent half. Do not relitigate the adults-only / minors-exclusion rule ? it is a ratified guardrail (CLAUDE.md).",
        "evidence": "README §2 D10 (line 37: free-tier data-for-reels, adults-only, derived-data+auto-delete, ToS counsel-reviewed, live training gated to M9), §6 line 106 (DPA/consent remains owner/counsel-gated before public non-owner serving), §7 M9 row (line 122 'DPA/consent gate wiring', Gated ? owner privacy/counsel), §7 M11 (line 124, consent gate dependency); CLAUDE.md guardrails (paid-LLM never a training source; exclude minors; per-user training data needs separate explicit opt-in); 05-COST-ATTRIBUTION/06 ADR-011 carry-over 'DPA/consent for non-owner uploads' (README line 53)"
      },
      {
        "id": "m12-gdrive-oauth",
        "title": "M12 gdrive-api OAuth StorageBackend: codeable against a MOCKED Drive client NOW; needs the owner's Google OAuth client/creds to run for real",
        "milestone": "M12",
        "gate_type": "creds_calibration",
        "blocks": "The D14 Northstar Drive round-trip actually working against a real user's Drive (read source + write reel back next to it). The backend CLASS + its wiring into the StorageBackend ABC is not blocked; only real OAuth credentials + a real consent screen are.",
        "claude_doable_now": "The full GDriveApiStorage backend behind the existing seam, with a mocked Drive service. (1) A new GDriveApiStorage(StorageBackend) implementing fetch_to_local/put/exists/stat/list/delete (and signed_url for the share-back) against the Google Drive API ? distinct from the mount-only gdrive.py which explicitly does NOT talk to the

API (gdrive.py:1-21). Register it in _backend_class (storage/__init__.py:77-91) under a new scheme (e.g. gdrive-api or a flag on gdrive) and add the Literal to StorageConfig.backend/input_backend (models.py:304,315). (2) Lazy-import the googleapiclient/google-auth SDK exactly like gcs.py does (gcs.py:24) so `import backend.storage` stays cheap. (3) Unit-test it FULLY by injecting a fake Drive `service` object (the GCS backend already supports client injection for tests, gcs.py:18-25) ? assert fetch downloads a file-id, put creates a file in the parent folder, list/exists/stat map Drive metadata ? zero network, zero Google account. (4) Token plumbing that reads a per-user OAuth token from a path/secret-manager ref (NOT config, per the 'secrets never in config' rule, models.py:297-303, __init__.py:98), with the token supplied at runtime. (5) The 'write reel back next to the source' resolution logic (parent-folder of the source file-id) as a pure, testable function.",
        "owner_prerequisite": "A Google Cloud OAuth setup, not code: (a) a Google OAuth 2.0 CLIENT ID + client secret (a registered OAuth app / Google Cloud project with the Drive API enabled), (b) the OAuth consent screen configured + scopes chosen (drive.file vs full drive) + Google verification if going public, (c) the per-user OAuth refresh/access TOKENS obtained via a real consent flow (which itself ties to the M9 consent gate for non-owner users), (d) approval that Drive write-back to a user's Drive is in scope. None of (a)-(d) is needed to write or unit-test the backend against a mocked service.",
        "unblock_value": "medium",
        "recommendation": "Approve building GDriveApiStorage NOW against an injected/mocked Drive service (mirroring gcs.py's client-injection test seam), wired into _backend_class + the StorageConfig Literal, secrets read from a runtime token ref. Owner later supplies only the OAuth client creds + consent-screen config + per-user tokens to run it live. Recommend a NEW scheme name (gdrive-api://) to keep the mount-only gdrive:// backend (truly-local) untouched. Note the live per-user token flow overlaps M9-consent for non-owner users, so real-Drive write-back should not ship ahead of the consent gate.",
        "evidence": "gdrive.py:1-21 (mount-only, explicitly 'does NOT talk to the Drive API'); storage/base.py:17-49 (StorageBackend ABC + signed_url NotImplementedError default), gcs.py:18-25 (lazy SDK import + injectable client test seam), gcs.py:61-63 (signed_url impl to mirror); storage/__init__.py:77-91 (_backend_class registry to extend), :94-103 ('secrets never stored here'); models.py:304,315,328-331 (StorageConfig.backend/input_backend Literal + gdrive loose dict), models.py:297-303 ('secrets are never stored here'); README §2 D14 (line 41: gdrive-api OAuth backend, read source + write reel back), §8 #7 (line 139 RESOLVED->D14), §7 M12 row (line 125, Gated ? owner OAuth/consent)"
      }
    ]
  },
  {
    "area": "M11 data-flywheel harness + D10/D11 consent/quality gates ? the seam between Claude-doable default-OFF plumbing (capture?store?shadow-eval?goldenize-stub) and the M9/D10 consent + adults-only + counsel-ToS decision (owner).",
    "gates": [
      {
        "id": "m11-plumbing",
        "title": "M11 data-flywheel PLUMBING ? capture?store?shadow-eval?goldenize-stub, default-OFF, faked inputs",
        "milestone": "M11",
        "gate_type": "design_choice",
        "blocks": "Nothing downstream is blocked by THIS gate ? the plumbing is buildable now and merges dark (default-OFF). It only becomes a live data flywheel once M11-consent unblocks the consented-adult retention path. Until then it runs on faked/owner-own inputs and produces no user-data side effects.",
        "claude_doable_now": "A LOT, all default-OFF + mocked. (1) Capture/store step: a `flywheel.capture(video_id, workspace, artifacts)` helper that, when `config.storage.single_folder_per_video` is ON AND a new `flywheel.capture_enabled` flag is ON (default False), copies the already-produced source upload + Gemini reel + rally output/report into `workspaces/<video_id>/{source,reels,outputs}/` via the existing pure helper `backend/utils/workspace.py::for_video` (VideoWorkspace.source/reels/outputs, lines 74-95) + the StorageBackend ABC (`backend/storage/`); tests use the storage fakes. NOT wired into any core write site ? `single_folder_per_video` is still un-wired (only referenced in `backend/config/models.py:322`), so this lands as an opt-in capture path, no behaviour change when off. (2) Shadow-eval adapter: a `flywheel.shadow_eval` shim that ingests a Gemini rally output (intervals/report) and an owned-model rally output into the harness's shape and calls `backend/eval/experiment.py::compare` (line 485) / `compare_unlabeled` (line 536) to track the owned-vs-Gemini gap per job; pure/offline, unit-tested against synthetic intervals (the harness

is already pure+offline, `test_experiment.py`/`test_experiment_honest.py` exist). NOTE the real seam: `compare()` consumes `VideoCandidates` from persisted candidates, while a Gemini reel is a serving output ? so the codeable piece is the ingest adapter that maps both serving outputs into `VideoCandidates`/`Interval` lists; no new scoring math. (3) Goldenize-promotion STUB: reuse the EXISTING promote-if-served-safe gate `backend/eval/promotion.py::promote_if_served_safe` + `per_user_gate.should_promote_for_user` to decide eligibility, and write a `flywheel.promote_stub` that emits a golden-TRAINING-row candidate file (mirroring the offline `backend/eval/fusion_golden.py:149` `*_golden_features.json` sink) GATED behind a default-OFF flag + a `consent_attested=True` precondition that is hardcoded-False/faked in tests. (4) An inert `consent` placeholder field on the job/config (defaulting to no-consent/auto-delete, matching today) so the routing fork (consent?capture vs no-consent?delete) is wired and unit-tested with FAKE consent values ? the routing logic is codeable; only the real consent SOURCE is gated. (5) Tests + a TESTING_STRATEGY entry. All of this is one isolated PR off origin/master, default-OFF, mocked ? no GPU/cloud/owner needed.",
        "owner_prerequisite": "A design pick on the seam shape: do we (a) add the inert `flywheel.*` config flags + capture/shadow-eval/promote-stub helpers now as dark plumbing, accepting they no-op until M9/consent lands, or (b) defer all M11 code until M9 ships? Recommend (a). No spend, no creds, no counsel needed for the plumbing itself ? only the owner's OK to land dark default-OFF code that depends on a not-yet-wired `single_folder_per_video` and a faked consent field.",
        "unblock_value": "high",
        "recommendation": "Approve (a): land M11 as one isolated default-OFF PR now ? capture helper (reusing `workspace.py` + StorageBackend), a shadow-eval adapter over `experiment.compare`, and a goldenize promote-stub reusing `promotion.promote_if_served_safe` ? all behind `flywheel.*` flags defaulting OFF and a faked `consent_attested` precondition, with storage-fake + synthetic-interval tests. This builds the entire mechanical loop dark so that when M11-consent unblocks, flipping it on is config-only, not new code.",
        "evidence": "docs/COMPUTE_DECOUPLED_SERVING/README.md §4.6 lines 94-100 (capture/store/shadow-eval/goldenize design) + tracker M11 line 124; backend/utils/workspace.py:46-114 (pure per-video layout helper, default-OFF Phase-0, not yet wired); backend/config/models.py:317-327 (single_folder_per_video=False, workspace_root/prefix ? config exists, un-wired); backend/eval/experiment.py:485 compare() / :536 compare_unlabeled() / :625 load_video_candidates (pure offline A/B over VideoCandidates ? the shadow-eval engine); backend/eval/fusion_golden.py:148-156 (the only golden-row writer today, OFFLINE extractor ? the goldenize sink to mirror); backend/eval/promotion.py:78-84 promote_if_served_safe (the served-safe promotion gate to reuse); grep confirms NO production goldenize/promotion-to-training write path and NO consent field wired anywhere."
      },
      {
        "id": "m11-consent",
        "title": "M11 data-flywheel CONSENT/RETENTION decision ? D10 adults-only + derived-data + counsel-ToS, gated to M9",
        "milestone": "M11",
        "gate_type": "counsel_consent",
        "blocks": "Blocks FLIPPING the M11 plumbing live: the consented-retention path (keep raw video in the training pool vs no-consent auto-delete), the adults-only training carve-out, and any real promotion of an actual user-uploaded video into the golden TRAINING set. Until this is granted, the M11 plumbing may only run on owner-own/faked footage and must default to the no-consent auto-delete branch. Also blocks public, non-owner serving generally (DPA).",
        "claude_doable_now": "Almost nothing on the DECISION itself ? but the code that CONSUMES the decision is the m11-plumbing gate above and is doable now (the consent field can be wired inert/faked, defaulting to no-consent/delete, so the fork is tested before the real source exists; gt_loader.py:11 already anticipates a 'future consent fingerprint'). What Claude CANNOT write without the owner/counsel: the actual consent-capture UX/ToS copy, the adults-only attestation mechanism, the legal text defining the data-for-reels trade, the retention/deletion policy values, and the decision to route real uploaded raw video into the training pool. These are policy + legal artifacts, not code.",
        "owner_prerequisite": "Owner + counsel must (1) finalize the D10 ToS / free-tier data-for-reels trade as counsel-REVIEWED text; (2) decide the adults-only attestation mechanism and that minor footage is reel-only (no training pool); (3) confirm derived-data-only + auto-delete-raw retention policy and the consented-retention bucket; (4) land the M9 edge-auth + per-tenant + consent-gate wiring this depends on (M11 depends on M7,M9). This is a counsel/consent + privacy decision, explicitly owner-gated; live-training-on-uploads is gated to M9/public-signup.",
        "unblock_value": "medium",

```
        "recommendation": "Keep owner/counsel-gated; do NOT attempt in code. Owner action: get
the D10 ToS counsel-reviewed and pick the adults-only attestation mechanism, THEN flip the
(already-built, dark) M11 plumbing live by enabling the consent source +
`flywheel.capture_enabled`. The value is medium-for-Claude because almost all the mechanical
loop is independently buildable behind the faked consent field (see m11-plumbing); this gate
frees only the final config flip + real-data routing, not new Claude PRs.",
        "evidence": "docs/COMPUTE_DECOUPLED_SERVING/README.md D10 line 37 (adults-only +
derived-data + ToS-counsel-reviewed + live-training gated to M9) + D12 line 38 + §4.6 line 100
('adults-only; raw-retention gated by the D10 consent') + §6 line 106 ('DPA/consent gate ?
remain owner/counsel-gated before public, non-owner serving'); tracker M11 line 124 Gated='?
owner (consent/privacy)', Depends=M7,M9; M9 line 122 ('DPA/consent gate wiring', ? owner
privacy/counsel); backend/eval/gt_loader.py:11 names a 'future consent fingerprint' (field not
yet built); grep confirms no consent/attestation field is wired in backend/ today."
      }
    ]
  },
  {
    "area": "Cross-track gates + housekeeping outside the M-series (COMPUTE_DECOUPLED_SERVING
serving track) ? what blocks/frees independent Claude work vs what needs the owner",
    "gates": [
      {
        "id": "p3-workspace-reconcile-close",
        "title": "Reconcile PER_VIDEO_WORKSPACE P3 with shipped M3 output-base + close the doc",
        "milestone": "per-video-workspace P3 / housekeeping",
        "gate_type": "housekeeping",
        "blocks": "Nothing hard-blocks, but the doc currently double-claims the 'de-hardcode
output/' win, so the P3 scope is ambiguous and the §7 tracker is stale. Clean reconcile lets
P1->P6 proceed with a correct P3 definition.",
        "claude_doable_now": "YES ? fully Claude-doable doc PR, NO owner needed. VERIFIED: M3
(#282) added backend/utils/paths.py::output_base(config) (paths.py:6-19) and rerooted the chunk
dirs under it (trajectory_hybrid.py:314, yolo_hybrid.py:269 and :408, gemini.py:540). BUT the M3
claim that it 'kills hardcoded output/' is only PARTLY true: the BASE is now configurable, yet
the folder NAMES 'chunks_{provider}_handoff' and 'chunks_yolo' are STILL hardcoded literals
(yolo_hybrid.py:408 os.path.join(output_base, 'chunks_yolo'); trajectory_hybrid.py:314
f'chunks_{provider}_handoff'). So M3 SUBSUMED the 'de-hardcode the output/ base' half
(PER_VIDEO_WORKSPACE.md:50 + :85 P3 row), but did NOT do the per-video tmp/ convergence P3 still
owns. Claude PR: (1) edit PER_VIDEO_WORKSPACE.md §3 line 50 + §7 P3 row (line 85) + §10 anchors
(line 107, which still cites trajectory_hybrid.py:237,313 ? now :314) to mark the output-base
half DONE-via-M3 and re-scope P3 to ONLY the tmp/ + detector/ routing through workspace.py; (2)
cross-link M3 (#282) and paths.output_base; (3) update DOC_STATUS.md:38 + NEXT_STEPS.md:16
wording. Pure doc edits, no code, no tests needed.",
        "owner_prerequisite": "None. (Optional, non-blocking: owner may later confirm §8.5
video_slug-vs-MD5 naming, but the reconcile does not need it.)",
        "unblock_value": "medium",
        "recommendation": "Ship a single doc-only PR that flips the M3-subsumed output-base half
of P3 to DONE, re-scopes the remaining P3 to the per-video tmp/+detector/ routing, and fixes the
stale :237,:313 line anchors. No owner action.",
        "evidence": "backend/utils/paths.py:6-19 (output_base);
backend/pipeline/segmenters/yolo_hybrid.py:408 'chunks_yolo' literal, :269 handoff;
trajectory_hybrid.py:314 'chunks_{provider}_handoff'; gemini.py:540;
docs/PER_VIDEO_WORKSPACE.md:50, :85 (P3 row), :107 (stale anchors);
docs/COMPUTE_DECOUPLED_SERVING/README.md:116 (M3 done); docs/DOC_STATUS.md:38;
docs/NEXT_STEPS.md:16"
      },
      {
        "id": "ops-agent-codify-deploy-gcp",
        "title": "Codify GCP Ops-Agent (observability) into deploy/gcp provisioning + VM
startup-script",
        "milestone": "housekeeping / cross-track infra (NEXT_STEPS P1 #6)",
        "gate_type": "housekeeping",
        "blocks": "Every new GPU VM today needs 3 manual console/CLI steps for observability;
missing them = empty GPU graphs (the owner-flagged failure that happened twice). Codifying makes
observability automatic on M5/M6 VMs.",
        "claude_doable_now": "YES ? Claude-doable infra-script PR, NO owner needed (no spend;
scripts only run when the owner later provisions). VERIFIED: deploy/gcp/provision.sh does
```

storage+IAM+budget ONLY (it enables compute/billing/iam APIs at :23-25 but NOT logging/monitoring, and grants only roles/storage.objectAdmin at :34-36). The 3 Ops-Agent pieces live solely as README prose in §3b (README.md:119-127): (1) gcloud services enable logging.googleapis.com monitoring.googleapis.com; (2) add roles/monitoring.metricWriter + roles/logging.logWriter to rally-worker SA; (3) curl add-google-cloud-ops-agent-repo.sh | bash --also-install on the box. README.md:129 itself says 'Best: bake ... into the VM --metadata=startup-script'. Claude PR: add the 2 services + 2 SA roles into provision.sh (idempotent, matches the existing pattern at :23-25/:34-36); add a deploy/gcp/ops-agent-startup.sh (the curl|bash one-liner) and wire --metadata-from-file startup-script=ops-agent-startup.sh into the README §3 create command (and/or a create-vm.sh wrapper). All offline-authorable; the owner just runs them next provision. No GPU/cloud run needed to write+merge.",
        "owner_prerequisite": "None to write/merge. Owner only needs to RUN the updated provision.sh / create command on the next real VM (that run is M5/M6 owner-GPU/spend territory, not this PR).",
        "unblock_value": "medium",
        "recommendation": "Single deploy/gcp PR: extend provision.sh (logging+monitoring APIs + metricWriter/logWriter roles) and add a startup-script baked into the VM-create command. Ships ready; owner runs it when provisioning.",
        "evidence": "deploy/gcp/provision.sh:22-25 (APIs, no logging/monitoring), :34-36 (only storage.objectAdmin); deploy/gcp/README.md:113-130 (§3b Ops-Agent manual steps + 'bake into startup-script' note); memory gcp-ops-agent-enable (owner directive, missed twice); docs/NEXT_STEPS.md:17 (P1 #6)"
      },
      {
        "id": "doc-hygiene-completed-stubs",
        "title": "Confirm/finish the COMPLETED top-level pointer stubs (DOC_STATUS §4 finding #4)",
        "milestone": "housekeeping / doc-lifecycle",
        "gate_type": "housekeeping",
        "blocks": "Nothing functional; it's a doc-cleanliness loose end (two copies of three hardening docs).",
        "claude_doable_now": "YES ? Claude-doable doc PR, NO owner needed. VERIFIED: docs/DEFERRED_HARDENING_PLAN.md (11 lines), CODE_AUDIT_AND_TEST_HARDENING.md (13 lines), HARDENING_LOOP_HANDOFF.md (15 lines) are intentional short pointer-stubs; full copies exist under docs/archives/*-COMPLETED-2026-06-14.md. Per #165 these are deliberate. Claude PR: either (a) add a one-line 'intentional pointer-stub (per #165)' header to each + flip DOC_STATUS.md:70 / §4 finding #4 to RESOLVED, or (b) finish-archive (delete the stubs, repoint inbound refs in CLAUDE.md/README) ? option (a) is the lower-risk default. Pure doc edits.",
        "owner_prerequisite": "None ? but a 1-line owner preference (keep stubs vs single-copy) would settle option (a) vs (b). Defaulting to (a) needs no owner.",
        "unblock_value": "low",
        "recommendation": "Default to option (a): stamp the three as intentional pointer-stubs and mark DOC_STATUS §4 #4 RESOLVED. No owner action.",
        "evidence": "docs/DEFERRED_HARDENING_PLAN.md (11 ln), CODE_AUDIT_AND_TEST_HARDENING.md (13 ln), HARDENING_LOOP_HANDOFF.md (15 ln) ? stubs; docs/DOC_STATUS.md:70, :87 (finding #4); archived full copies docs/archives/*-COMPLETED-2026-06-14.md"
      },
      {
        "id": "doc-hygiene-hosting-plan-banner",
        "title": "Add the ADR-013/site-overtaken currency banner to HOSTING_PLAN.md (DOC_STATUS §4 #5)",
        "milestone": "housekeeping / doc-lifecycle",
        "gate_type": "housekeeping",
        "blocks": "Nothing; a stale PROPOSAL doc can mislead a cold reader (still lists 'All-DigitalOcean' compute as live).",
        "claude_doable_now": "YES ? Claude-doable doc PR, NO owner needed. VERIFIED: DOC_STATUS.md:58 + §4 finding #5 (:88) flag HOSTING_PLAN.md as 'PROPOSAL but partly overtaken by ADR-013 (DO dropped) + the shipped khelsutra site'; the UI_PROPOSAL.md banner is already ? done (:59,:88) but the HOSTING_PLAN banner is 'still pending'. HOSTING_PLAN.md:4 still frames DigitalOcean as a live constraint. Claude PR: add a top currency banner pointing to ADR-013 (GCP sole compute) + the shipped site, matching the UI_PROPOSAL precedent; flip DOC_STATUS §4 #5 to RESOLVED. Pure doc edit.",
        "owner_prerequisite": "None.",
        "unblock_value": "low",
        "recommendation": "One doc PR adding the currency banner to HOSTING_PLAN.md (mirror the

```
UI_PROPOSAL banner) and closing DOC_STATUS §4 #5. No owner action.",
        "evidence": "docs/HOSTING_PLAN.md:4 (DigitalOcean still framed live);
docs/DOC_STATUS.md:58, :88 (finding #5, 'HOSTING_PLAN banner still pending', UI_PROPOSAL already
?)"
      },
      {
        "id": "doc-hygiene-archive-relocation",
        "title": "Field/PMF/Roora archive + data-layer docs/ dir relocation (DOC_STATUS §4 /
NEXT_STEPS line 31)",
        "milestone": "housekeeping / doc-lifecycle",
        "gate_type": "housekeeping",
        "blocks": "Nothing; the bulk was already done 2026-06-21 ? only a boundary confirm
remains.",
        "claude_doable_now": "MOSTLY DONE; a thin Claude-doable remainder, NO owner needed for
the mechanical part. VERIFIED: DOC_STATUS.md:83 shows the field-ops->archives/field-pmf,
Roora->archives/roora-vendor, and data_pipeline/ carve are ALREADY done with links rewired.
NEXT_STEPS.md:31 lists the residual 'field/PMF/Roora -> archives + a data-layer docs/ dir (owner
boundary confirm)'. The mechanical archive/move is Claude-doable per the §2 due-diligence
checklist; the ONLY owner-gated bit is the 'data-layer boundary confirm' (a design/scope call on
what counts as no-ML data-layer). Claude can do everything except that confirm.",
        "owner_prerequisite": "A one-line owner confirm of the data_pipeline/ 'no
segmentation/stitching' boundary (a scope/design pick). Everything else (the moves, link
rewires, README boundary note) is already done or Claude-doable.",
        "unblock_value": "low",
        "recommendation": "Treat the mechanical relocation as DONE (it is); surface ONLY the
data-layer-boundary line for a quick owner yes/no, then Claude closes the NEXT_STEPS line-31
residual.",
        "evidence": "docs/DOC_STATUS.md:83 (relocation done, links rewired, 3 dir READMEs), :44
(data_pipeline LIVING no-ML); docs/NEXT_STEPS.md:31 (residual + 'owner boundary confirm')"
      },
      {
        "id": "do-droplet-retire",
        "title": "Retire the shared DO droplet claude-do-rally-test",
        "milestone": "housekeeping (owner-action) / NEXT_STEPS line 28",
        "gate_type": "run_execution",
        "blocks": "~$24/mo standing cost on a shared testbed that ADR-013 made obsolete (GCP is
sole compute). Also a prerequisite to retiring the DO token (next gate).",
        "claude_doable_now": "NONE on the engine side ? this is a live cloud resource the owner
controls. No code/PR here unblocks it (the deploy/digitalocean/* scripts are already marked
DEPRECATED, README.md:3). The droplet is explicitly 'another session's testbed', owner-
coordinated cleanup. Claude can at most leave a tracking note; it cannot doctl-delete an account
resource it has no creds for and must not.",
        "owner_prerequisite": "Owner deletes the DO droplet claude-do-rally-test via the DO
console/doctl (coordinating with the other session that uses it as a testbed).",
        "unblock_value": "low",
        "recommendation": "Owner: delete claude-do-rally-test once the sibling session no longer
needs it; it is pure standing cost post-ADR-013.",
        "evidence": "docs/archives/decisions/DECISIONS.md:681 (droplet owner-coordinated
cleanup, token retire-after); docs/NEXT_STEPS.md:28; memory gcp-vm-always-delete (DO droplet
~$24/mo shared testbed); deploy/digitalocean/README.md:3 (DEPRECATED)"
      },
      {
        "id": "do-token-retire",
        "title": "Retire the DigitalOcean API token",
        "milestone": "housekeeping (owner-action) / NEXT_STEPS line 28",
        "gate_type": "creds_calibration",
        "blocks": "A live DO Personal Access Token (read+write Droplets/Spaces) is an unused
standing credential post-ADR-013; leaving it is a secret-hygiene risk.",
        "claude_doable_now": "NONE ? credential retirement is owner-only (the token lives in the
DO console, Claude has no access and must not). Depends on the droplet being gone first. Claude
can only document the dependency.",
        "owner_prerequisite": "Owner revokes the freshly-rotated DO token in the DO console (DO
-> API -> Tokens) AFTER the claude-do-rally-test droplet is deleted.",
        "unblock_value": "low",
        "recommendation": "Owner: revoke the DO token once the droplet is deleted; both are pure
```

post-ADR-013 cleanup, no engine dependency.",
        "evidence": "docs/archives/decisions/DECISIONS.md:681 ('freshly-rotated DO API token can be retired once that droplet is gone'); deploy/digitalocean/README.md:25 (the DO token scope); docs/NEXT_STEPS.md:28"
      },
      {
        "id": "repo-rename-off-badminton",
        "title": "Rename the repo off 'badminton'",
        "milestone": "housekeeping (owner-action) / BACKLOG ?",
        "gate_type": "design_choice",
        "blocks": "External sharing/branding (the project is multisport); touches the remote name + README/CLAUDE.md framing + any hardcoded GitHub URLs. Owner-emphatic 'ASAP' before broad external sharing.",
        "claude_doable_now": "PARTIAL but NOT safely without the owner's chosen name. Claude CANNOT rename the GitHub remote (owner/admin action). Claude COULD, ONCE the owner picks a name, do a follow-up PR sweeping in-repo framing (README/CLAUDE.md prose, and any hardcoded github.com/Khelsutra/badminton-highlight-indexer URLs ? e.g. the PR links in COMPUTE_DECOUPLED_SERVING/README.md:114-117). But with no chosen name there is nothing concrete to write now; doing a guessed name would be wrong. So: NONE until the owner picks the new name.",
        "owner_prerequisite": "Owner (a) picks the new repo name and (b) renames the GitHub repo (admin). DECISIONS.md:424 floats 'sports-temporal-models' as a candidate but it is NOT finalized.",
        "unblock_value": "medium",
        "recommendation": "Owner: pick + apply the new repo name (candidate 'sports-temporal-models' per ADR). THEN Claude does a one-PR in-repo sweep of prose + hardcoded URLs. Until the name is picked, nothing is Claude-doable.",
        "evidence": "docs/BACKLOG.md:13 (? owner-emphatic ASAP, touches remote+README+URLs); docs/archives/decisions/DECISIONS.md:424 ('sports-temporal-models, to be finalized with the pending repo rename'); docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:159; docs/NEXT_STEPS.md:27"
      },
      {
        "id": "wasb-c3-weight-provenance",
        "title": "WASB-C3 weight-provenance resolution (gates SHARING a trained model)",
        "milestone": "P2 #13 / commercial-licensing gate",
        "gate_type": "counsel_consent",
        "blocks": "Sharing/distributing a WASB-derived trained model commercially (the cloud-serving + any owned-weights distribution path). Multiplies per sport. A clean head does NOT launder the underlying academic-checkpoint provenance.",
        "claude_doable_now": "NONE in this repo by design ? it is an explicit 'no code change until resolved upstream' tracking-only item. Claude can only keep the tracking docs honest (e.g. ensure COMPUTE_DECOUPLED_SERVING honest-limits §6 keeps citing it, which it already does at README.md:106). The resolution is legal/sourcing, not code.",
        "owner_prerequisite": "Owner + counsel resolve the provenance: either confirm the WASB-SBDT checkpoint's commercial-distribution terms, OR train/swap to own clean weights. Until then the loader/gate refuses commercial ship regardless of F1 (DECISIONS.md:418-422).",
        "unblock_value": "low",
        "recommendation": "Owner/counsel: resolve WASB checkpoint provenance (confirm terms or own-weights/clean-swap). No engine PR is blocked OR enabled by it now ? it gates the eventual model-SHARING step only.",
        "evidence": "docs/COMMERCIALIZATION.md:65 (weights provenance open, C3), :74; docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:121, :156; docs/RALLY_DETECTION_QUALITY_REPORT.md:366; docs/archives/decisions/DECISIONS.md:418-422 (gate refuses ship while C3 unresolved); docs/COMPUTE_DECOUPLED_SERVING/README.md:106; docs/POST_166...RISK_ASSESSMENT.md:64 ('No code change in this repo until resolved upstream')"
      },
      {
        "id": "collector-md5-export",
        "title": "Collector md5 export / keying (cross-repo, sports-data-collector)",
        "milestone": "P2 #15 / cross-repo data contract",
        "gate_type": "design_choice",
        "blocks": "Robust train/eval split keying + workspace join across the collector<->indexer boundary; the collector keys by human video_id while the indexer keys by file MD5, so re-encodes evade dedup and a curate export 'split' field is needed.",
        "claude_doable_now": "NONE in THIS repo ? the change lives in the sibling sports-data-

```
collector repo (curate export must emit md5/split). The indexer side that consumes it is ALREADY
built and pure-logic-tested: backend/eval/splits.py::load_split + assert_disjoint (BACKLOG.md:49
'DONE ? pure-logic core, PR #171'). So the indexer half needs no owner; the BLOCKING half is a
cross-repo collector change Claude cannot make from this checkout.",
        "owner_prerequisite": "Owner (or a collector-repo session) adds the md5 +
split:'train'|'eval' field to the collector's curate export manifest (the ADR-002 cross-repo
contract). A dedicated design session is flagged needed.",
        "unblock_value": "low",
        "recommendation": "Owner: schedule the collector-side curate export change (md5 + split
field). The indexer consumer (splits.py) already exists, so once the collector emits it, only a
thin wire-up PR remains here.",
        "evidence": "docs/CODE_MAP.md:275 (indexer keys by file MD5, collector by human
video_id), :277; docs/BACKLOG.md:49 (splits.py DONE PR #171; STILL DEFERRED = collector curate
export split field + wire-in; 'needs a collector-side manifest change');
docs/archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:68-72;
docs/COMPUTE_DECOUPLED_SERVING/README.md:53 (carries collector md5 keying gate)"
      }
    ]
  }
]

Produce a markdown report with these sections:
1. **The one cross-cutting decision** ? state the single highest-leverage choice the owner can
make: "for gated milestones whose CODE is writeable now behind default-OFF + mocked tests,
should Claude write+merge the code now and defer only the real run/spend/calibration/counsel to
you?" Explain how big an unblock a blanket YES is (which milestones it frees) vs the risk
(merging code that can't be run-verified locally).
2. **Decision menu, ranked by unblock value** ? a table: Gate | Type | What's blocked | Claude-
doable NOW (default-OFF code) | Owner must provide/decide | Unblocks | Recommendation. Order
HIGH unblock_value first. De-duplicate gates that multiple readers reported (e.g. M9 split into
auth-code vs consent-counsel).
3. **Immediately independent (no owner needed)** ? the PRs Claude can start RIGHT NOW with zero
gating (M5 startup-check code, M8 ledger code w/ placeholder prices, M11 plumbing, P3 reconcile,
Ops-Agent codify). Give a suggested merge order.
4. **Genuinely owner-only (cannot be pre-coded)** ? the short list that truly needs the owner
first (real GPU run for the M5 runlog, GCP spend for M6/M7 runs, counsel for DPA/consent ToS,
OAuth client creds, real GCP prices, repo rename, droplet/token retirement).
5. **The cheap design picks** ? §8 Q2/Q8/Q9/Q10 + any others: one-line each with a recommended
default, so the owner can rubber-stamp them in one pass.

Be concrete and honest. Distinguish "writeable + mock-tested now" from "verifiable only by the
owner". Keep it tight ? this goes straight to the owner to decide from.
```

2. assistant (2026-06-21T09:27:57.682Z)

I have all the structured findings from the 5 grounded readers. Let me synthesize them into the prioritized decision menu. Let me verify a couple of the load-bearing claims before finalizing, since several gates hinge on the same code facts.

3. user (2026-06-21T09:27:58.724Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
```

```
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
```

4. user (2026-06-21T09:27:59.616Z)

```
110
111     | ID | Deliverable | Depends | Gated? | Status | PR |
112     |----|-------------|---------|--------|--------|----|
113     | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |
```

? APPROVED 2026-06-21 |
114 | M1 | `deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? | [#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
115 | M2 | `input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? | [#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
116 | M3 | Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? | [#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
117 | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283) |
118 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |
119 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | ? |
120 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
121 | M8 | Per-job **cost ledger** (v_ migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | ? |
122 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
123 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |
124 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
125 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
126
127   **Testing strategy is a first-class deliverable** ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).
128
129   ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
130   **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
131   1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
132   3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
133   4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
134   5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
135   6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).
136
137   **Still open (non-gating, smaller):**
138   2. **Stitch placement:** always co-located in the detect job (simplest, one metered unit) vs split to a cheap CPU Cloud Run service when bursty? *Default co-located unless metrics say otherwise.*
139   7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
140   8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264 (determinism/parity)?
141   9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and

```
route larger files elsewhere?
142     10. **Directory name:** `COMPUTE_DECOUPLED_SERVING/` (chosen) vs keep
`CLOUD_RUN_GPU_SERVING/` ? confirm.
143
144     ## 9. Definition of done
```

5. assistant (2026-06-21T09:29:35.954Z)

```
The findings are well-grounded and consistent with the doc. I have what I need to synthesize.
```

Decision Menu ? COMPUTE_DECOUPLED_SERVING Serving Track (M5?M12 + §8 + h

1. The one cross-cutting decision

```
**The single highest-leverage choice:** *For every gated milestone whose CODE is writeable now
behind `default-OFF` + mocked/faked tests, authorize Claude to write + merge the code now, and
reserve your gate strictly for the part that genuinely needs you ? the real GPU run, the GCP
spend, the calibrated prices, the counsel-reviewed ToS, the OAuth creds.*

A blanket **YES** is enormous: it splits **every remaining gate but one** into an immediately-
mergeable code PR + a thin owner residual. It frees the code surface of **M5** (preset +
startup-checks + L4 parity test), **M6** (the whole ComputeBackend dispatch shim + Dockerfile ?
the biggest single independent chunk the cloud track needs), **M8** (full ledger schema +
pricebook + aggregation + cost API), **M9-auth** (Cf-Access JWT verifier + per-tenant scoping ?
finally making the v4/v5 `user_id` columns load-bearing), **M11-plumbing**
(capture?store?shadow-eval?goldenize-stub), and **M12** (the `gdrive-api` StorageBackend). After
a YES, the *only* things still genuinely waiting on you are listed in §4 ? and they collapse to
**one real-GPU run, one batched GCP spend authorization, one counsel review, and one OAuth-app
registration**.

**The risk, stated honestly:** this is code that **cannot be run-verified locally** ? no GPU, no
cloud, no real billing/Drive/CF here. Mocked-dispatch tests prove the *control flow and
contracts*, not that the image actually runs on an L4, that the pricebook reconciles against a
real GCP invoice, or that a real Cf-Access JWT verifies. The doc itself codifies this (`README
§6`: green CI proves plumbing, **not** field quality or real-GPU detection). **Mitigations
already baked into the discipline:** everything lands at `profile=''` / `compute_target` default
= byte-identical to today (`presets.py:14`, the "profile='' == today" invariant), one isolated
PR per milestone off `origin/master`, additive-NULLable migrations (NULL = local), and the L4
profile-parity test as the load-bearing in-process proof. **Net:** the merged code carries near-
zero behavior risk to the shipped local path; the residual risk is purely "the real run may
surface integration gaps" ? which is exactly what the owner-run M5/M7 runlogs exist to catch.
**Recommendation: blanket YES.**

---
```

2. Decision menu, ranked by unblock value

```
De-duplicated across all 5 readers. M6/M9/M11 each had two readers; merged into the canonical
split (code-half vs owner-half).

| Gate | Type | What's blocked | Claude-doable NOW (default-OFF) | Owner must provide/decide |
Unblocks | Recommendation |
|------|------|----------------|---------------------------------|---------------------------|-
---------|----------------|
| **M6-code** ? Cloud Run dispatch shim + worker image | design pick (then merge) | M7, and
transitively M8/M9/M11/M12 | **YES, large.** New `backend/compute/` registry (ComputeBackend ABC
+ LocalCompute=identity default-ON + CloudRunCompute behind `compute_target=cloud_run`, default-
OFF), injectable Cloud Run client (mirror `gcs.py:18-25` DI), reuse `JobWaiting`?`claim_due_job`
park/resume verbatim, Dockerfile (ffmpeg+CUDA+WASB+fonts), mocked-dispatch + stub-image-smoke
tests. Worker entrypoint `python -m backend.worker infer` already exists. | A design pick (Cloud
Run **Jobs** vs Services-with-GPU; bucket-poll vs callback) + go-ahead to write. **No spend to
LAND.** | M7 e2e | **APPROVE now.** One PR, default-OFF. Recommend Cloud Run **Jobs**, stitch
co-located (D4). Single biggest independent cloud chunk. |
```

M8 ? cost ledger + pricebook + cost API	creds/calibration	M10; the DoD "cost visible per job"	**YES, nearly all.** v6 migration (`owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json` + `pricebook` table), all additive-NULL (mirrors v4 `user_id`, `database.py:219-237`); USD-compute + INR display at configurable FX; aggregation + `/api/jobs/{id}/cost` + `/api/videos/{id}/cost`; worker timing hooks; unit tests on a **seeded placeholder pricebook (`version='placeholder-v0'`, all rates 0.0)** ? $0-valued, nothing user-facing changes.	**Real calibrated GCP prices** (L4/T4 $/GPU-s, egress $/GB, storage GB-hr) + **FX rate + margin policy** + reconcile against a real GCP invoice on the first M7 run + flip-to-live OK. (`gemini_cost_usd` stays NULL placeholder ? `gemini.py` metrics are timing-only.)	M10 design; live billing	**APPROVE now** behind the zero pricebook. Owner's only later action: a one-row real-rate INSERT + FX/margin pick + flip.
M9-auth ? Cf-Access JWT verify + per-tenant scoping	creds/calibration	Multi-tenant identity being *trusted* in prod	**YES, the whole verify+scope half.** Add `PyJWT[crypto]` (no JWT dep today). FastAPI dependency: extract `Cf-Access-Jwt-Assertion`, fetch+cache team JWKS, verify sig/exp/iss/aud ? `user_id`. Unit-test **fully offline** with an in-test RSA keypair + fake JWKS (accept/reject: bad sig, expired, wrong aud, missing header). Thread `user_id` into `create_job`/queries (v4 columns already exist). Gate behind `security.edge_auth_enabled=False` (off = byte-identical to today). Tighten the placeholder CORS (`main.py:40-41`).	**Real Cloudflare config values:** team-domain/JWKS issuer URL, app AUD tag, and the **ingress-lock deploy step** (lock Cloud Run to the CF proxy so the header can't be spoofed) ? needs M7 live.	M10, M11, M12, public serving	**APPROVE now**, default-OFF. Makes v4/v5 `user_id` columns finally load-bearing without touching the local path. Run counsel review (below) in parallel ? auth must NOT wait on consent.
M5-code ? truly-local preset + startup checks + L4 parity test	run (then merge code)	M6 (control), the local?cloud switch story	**YES, most of M5.** Lock the truly-local preset coherently (detector/stitch-in-process/`storage=local`, `presets.py:44`); per-profile **startup checks** that fail/warn-fast (cloud profile without GCS/GPU ? loud error) ? pure config validation; the **L4 profile-parity test** (in-process truly-local vs worker+fake-gcs ? byte-identical windows+segment ranges); L1?L3 coverage. All $0, deterministic.	**One real-GPU run** on the owner's box ? committed `runlogs/` truly-local runlog (rally counts + ffmpeg smoke + 5-frame contact sheet).	M6 code	**APPROVE as TWO PRs:** Claude lands preset+checks+parity+coverage now; owner does one GPU run for the runlog.
M11-plumbing ? capture?store?shadow-eval?goldenize-stub	design pick (then merge)	Nothing downstream ? merges *dark*	**YES, the whole mechanical loop.** `flywheel.capture` reusing `workspace.py::for_video` (`:74-95`) + StorageBackend; `flywheel.shadow_eval` adapter mapping Gemini+owned outputs into `VideoCandidates`/`Interval` then calling `experiment.compare` (`:485`); `flywheel.promote_stub` reusing `promotion.promote_if_served_safe` (`:78-84`); an **inert faked `consent_attested` field** defaulting to no-consent/auto-delete so the fork is tested before the real source exists. All behind `flywheel.*` flags = OFF, storage-fake + synthetic-interval tests.	Just the OK to land dark default-OFF code (depends on the not-yet-wired `single_folder_per_video` + a faked consent field). No spend/creds/counsel for the **plumbing**.	Config-only flip once M9-consent lands	**APPROVE now.** Builds the entire loop dark so going live is config-only, not new code.
M12 ? `gdrive-api` OAuth StorageBackend	creds/calibration	D14 Northstar Drive round-trip; khelsutra app flow	**YES, the backend class.** New `GDriveApiStorage(StorageBackend)` (fetch/put/exists/stat/list/delete + `signed_url` share-back) against an **injected fake Drive service** (mirror `gcs.py:18-25`); register a new `gdrive-api://` scheme in `_backend_class` + the `StorageConfig` Literal; "write reel back next to source" = pure parent-folder resolver; token read from a runtime secret ref (not config). Full roundtrip tests, zero Google account. Keep the mount-only `gdrive://` (truly-local) untouched.	**Google OAuth app**: client ID/secret, consent screen + scopes (`drive.file` vs full) + Google verification, per-user tokens via real consent flow (ties to M9-consent for non-owner users).	D14 Northstar; mobile/cloud-native flow	**APPROVE now** against the mocked service. Don't ship real write-back ahead of the consent gate.
M7 ? cloud-serving e2e on L4 + DEPLOY_REPORT	run + spend	M8 numbers, M9 live endpoint, M11 capture, M12 round-trip; the cloud DoD	**Almost none new** beyond M6/M2. Pre-stage offline: DEPLOY_REPORT template + a mock-driven sample-run harness (upload?poll?signed-URL shape, contact-sheet gen) so the owner run is one-shot.	**GCP spend + a real L4 run** (billing on, GPU quota ? `asia-south1` L4 often stocked-out, Singapore fallback), owner golden corpus.	M8/M9/M11/M12 real data	**HOLD for owner spend; pre-stage everything.** Sequence smallest viable run (one short clip, cheapest-adequate GPU, **DELETE** after). Do not fake green.
M9-consent ? DPA/ToS + adults-only + retention	counsel/consent	Live train-on-uploads; M11 raw-retention; public non-owner serving	**Only the inert mechanism:** NULLable `consent_status`/`is_adult_attested`/`retention_class` (folds into the v6 migration), defaulting to the **safe no-consent path** (reel yes, training NO, raw auto-deleted); the capture-vs-delete

branch keyed on it. **Cannot** write ToS/DPA text, define valid attestation, or approve the training-retention flip. | **Counsel-reviewed ToS/DPA**, the legally-sufficient adults-only/minor-exclusion rule, retention policy text, and the flip-to-live approval. | M11 live capture | **Keep owner+counsel-gated.** Build only the default-OFF no-train mechanism. Schedule counsel review **in parallel** with M9-auth. Don't relitigate minors-exclusion (ratified guardrail). |
| **M10** ? `byo_worker` (design-only) | housekeeping | Nothing ? terminal, deferred | **~Nothing.** The enum/seam reservation is already done in M1. A premature build would violate the deferral. | Explicit owner approval to **un-defer at all** (product/strategy call; real ops burden). | Nothing | **Leave DEFERRED.** Spend no code budget. |

---

3. Immediately independent (no owner needed) ? start RIGHT NOW

These need **zero gating** ? pure default-OFF code or doc PRs, all $0, all mock/fixture-tested. Suggested **merge order** (dependency- and risk-ordered, one isolated PR each off `origin/master`):

1. **P3 doc reconcile** *(doc-only, lowest risk, clears stale anchors first)* ? flip the M3-subsumed output-base half of `PER_VIDEO_WORKSPACE.md` P3 to DONE-via-#282, re-scope remaining P3 to per-video `tmp/`+detector routing, fix stale `trajectory_hybrid:237,313` anchors (now `:314`), update `DOC_STATUS.md:38` / `NEXT_STEPS.md:16`.
2. **Ops-Agent codify** *(infra-script, no run)* ? extend `deploy/gcp/provision.sh` with logging+monitoring API enables + `metricWriter`/`logWriter` SA roles; add `ops-agent-startup.sh` and wire `--metadata-from-file startup-script=` into the create command. Ships ready; owner runs it next provision.
3. **Doc-hygiene batch** *(optional, fold into #1 or ship separately)* ? stamp the 3 hardening pointer-stubs as intentional (per #165) + flip `DOC_STATUS §4 #4`; add the ADR-013 currency banner to `HOSTING_PLAN.md` + flip `§4 #5`.
4. **M5 code PR** ? truly-local preset coherence + per-profile startup checks + L1?L4 coverage (incl. the parity test). *(Foundation for everything cloud; lands the parity proof.)*
5. **M6 code PR** ? `backend/compute/` ComputeBackend registry (LocalCompute default-ON, CloudRunCompute default-OFF) + injectable client + Dockerfile + mocked-dispatch/stub-image-smoke tests. *(Depends conceptually on M5's parity test as its control.)*
6. **M8 code PR** ? v6 migration + `pricebook` table (`placeholder-v0`, $0) + USD/INR aggregation + cost endpoints + worker timing hooks. *(Carries the shared `owner_id`/`user_id` jobs column M9 also needs.)*
7. **M9-auth code PR** ? `PyJWT[crypto]` + Cf-Access verifier + per-tenant `user_id` scoping + CORS tighten, behind `security.edge_auth_enabled=False`, RSA/fake-JWKS tests. *(Builds on the v6 jobs `user_id` column from #6.)*
8. **M11-plumbing PR** ? `flywheel.capture`/`shadow_eval`/`promote_stub` behind `flywheel.*`=OFF + faked `consent_attested`, storage-fake + synthetic-interval tests.
9. **M12 code PR** ? `GDriveApiStorage` + `gdrive-api://` scheme + fake-Drive-service roundtrip tests.

(Note: #6, #7, and the M9-consent inert columns all touch the same v6 migration ? coordinate so the migration is authored once, in #6, and #7/consent-columns extend it.)

---

4. Genuinely owner-only (cannot be pre-coded)

The short list that truly needs you first ? everything else above is Claude-mergeable now:

- **Real-GPU run ? the M5 truly-local runlog** (your box, your golden corpus). The one thing that flips M5 fully green.
- **GCP spend authorization (batched, scale-to-zero)** ? run M6's image on a real L4 (M7), capture the DEPLOY_REPORT. Smallest-viable first run; **DELETE** the service after (per the always-delete rule). One authorization unblocks the whole cloud-run-real path.
- **Counsel-reviewed ToS/DPA + adults-only attestation + retention policy** (M9-consent/D10) ? the only path to live train-on-uploads + M11 raw retention. Schedule **in parallel** with the M9-auth code; auth must not wait on it.
- **Real calibrated GCP prices + FX rate + margin policy** (M8) ? a one-row real-rate pricebook

INSERT + the flip-to-live OK + invoice reconcile on the first M7 run.
- **Google OAuth app** (M12) ? client ID/secret, consent screen + Drive scopes + Google verification + per-user tokens.
- **Cloudflare config values** (M9-auth runtime) ? team-domain/JWKS issuer + app AUD + the ingress-lock deploy step (needs M7 live). *(Code is writeable now; only these strings + the deploy step are owner-side.)*
- **Repo rename off "badminton"** ? pick + apply the new GitHub name (candidate `sports-temporal-models`, not finalized). THEN Claude sweeps in-repo prose + hardcoded URLs in one PR.
- **DO droplet `claude-do-rally-test` delete** (~$24/mo, obsolete post-ADR-013; coordinate with the sibling session) **? then revoke the DO API token.** No engine dependency.
- **WASB-C3 weight-provenance** (counsel/sourcing) ? gates *sharing* a trained model commercially, not any current engine PR. No code change in this repo until resolved upstream.
- **Collector md5/split export** (sibling `sports-data-collector` repo) ? the indexer consumer (`splits.py`, PR #171) already exists; only the collector-side manifest change is owner/cross-repo.
- **M10 un-defer** (`byo_worker`) ? explicit product approval to start it at all.

---

5. The cheap design picks (rubber-stamp in one pass)

One line each, recommended default in **bold** ? these are config-knob or one-word confirms, no spend/creds:

- **§8 Q2 ? Stitch placement:** **co-located in the detect job** (already D4; job table supports a future split non-breaking). Revisit only if M7 metrics show bursty compiles.
- **§8 Q8 ? NVDEC/NVENC vs libx264:** **libx264 for the stitch ENCODE** (protects byte-determinism that D5 + the L4 parity test depend on); optionally **NVDEC for DECODE only** (speed, no output-byte change). Defer NVENC-encode behind a config knob unless M7 cost data justifies relaxing parity.
- **§8 Q9 ? Input cap:** **hard reject at `<2GB` for v1** (matches the already-shipped M2 chunked-upload threshold), implemented as a config cap so raising to 4GB later is one line.
- **§8 Q10 ? Directory name:** **confirm `COMPUTE_DECOUPLED_SERVING/` final** (already the name everywhere + in ADR-014); drop `CLOUD_RUN_GPU_SERVING/`. Pure marker close.
- **M6 shim shape:** **Cloud Run Jobs (not Services-with-GPU)** for the detect+stitch unit ? matches the async upload?poll?reel shape + scale-to-zero (D7); **bucket-poll for completion** (reuses `JobWaiting`/`claim_due_job` verbatim).
- **Data-layer boundary** (`NEXT_STEPS:31` residual): a one-line yes/no on the `data_pipeline/` "no segmentation/stitching" boundary ? the only owner bit; everything mechanical is done/Claude-doable.

Bottom line: A single blanket **"write + merge the default-OFF code now, gate only the real run/spend/calibration/counsel"** turns 9 of the 10 remaining milestones into immediately-mergeable PRs (§3 gives the order) and shrinks your queue to **one GPU run, one batched GCP spend, one counsel review, one OAuth-app registration** (§4) plus six rubber-stamp design picks (§5). The only milestone with no meaningful code to write is M10 ? leave it deferred.

Key file anchors: `backend/config/presets.py:27,33,43`, `backend/infrastructure/database.py:219-237` (v4 additive-NULL pattern to mirror), `backend/storage/gcs.py:18-25` (DI test seam), `backend/worker/__main__.py:91-176` (existing container entrypoint), `backend/infrastructure/execution_policy.py` + `job_worker.py:41-152` (`JobWaiting`/park-resume), `backend/eval/experiment.py:485` + `promotion.py:78-84` (flywheel reuse), `backend/utils/workspace.py:74-95`, `backend/utils/paths.py:6-19` (M3 output-base), `deploy/gcp/provision.sh:22-25,34-36`. Doc: `docs/COMPUTE_DECOUPLED_SERVING/README.md` §7 tracker (`:111-125`), §8 (`:129-142`), TESTING_STRATEGY L4 parity proof.